

Framebuffer

Ein **Framebuffer** ist ein Bereich im RAM, **alle Pixel eines Bildes enthält** enthält, also das, was später auf dem Bildschirm angezeigt wird.

In der Vorstellung ist der Framebuffer wie ein grosses Array vor, dass für jeden Pixel eine Farbe gespeichert hat.

Beispiel bei 800x600

- Es gibt **800 x 600 = 480 Pixel**
- Für jeden Pixel wird z. B. ein 32 bit-Wert gespeichert(für RGBA)

Was passiert mit dem Framebuffer?

1. Dein Programm schreibt Farbdaten in den Ram (z. B. "Pixel 100,200 = rot")
2. Die **Grafik-Hardware liest den Framebuffer** regelmässig aus
3. Diese Daten werden auf den Monitor angezeigt

Wichtig: Der **Framebuffer** selbst ist nur **der Speicher**, nicht das Display selbst. Was du reinschreibst, entscheidet, was du siehst

Anwendungsfälle

- **Spielekonsolen**(z. B. Nintendo DS, GBA, PSP) – dort malst du direkt auf dem Bildschirm
- **Embedded-Geräte** – z. B. Bildschirme im Auto oder Mikrowellen-Menüs
- **Low-Level Programmierung** – wie bei Betriebssystemen, Bootloadern oder Baremetal-Code
- **Moderne Engines** – nutzen Framebuffer als Zwischenspeicher (z. B. für Schatten, Post-Processing, HDR)

Warum ist der Framebuffer so wichtig?

- Weil du damit **volle Kontrolle** über das Bild hast
- Du kannst **Pixel für Pixel bestimmen**, was angezeigt wird
- Du braucht keine Fenster, keine Buttons, kein GUI-System
- Ideal zum Lernen: Du verstehst, wie Grafik **wirklich funktioniert**

Wie sieht ein Framebuffer aus?

Ganz simpel gedacht:

```
uint32_t framebuffer[800 * 600]; // 32 Bit pro Pixel (RGBA)

framebuffer[0] = 0xFFFF0000; // Pixel oben links wird rot
framebuffer[1] = 0xFF00FF00; // Pixel daneben wird grün
```

Jeder Pixel ist nur eine Zahl – und die Zahl bestimmt die Farbe.

Kurzgesagt

Ein **Framebuffer** ist die einfachste Art, Bilder auf dem Bildschirm darzustellen. Statt Fenster, Icons oder komplexer Grafiksysteme schreibst du einfach **direkt Pixel in den Speicher** und das Ergebnis siehst du auf dem Display.

Das Prinzip ist alt, aber immer noch wichtig vor allem in der Spieleentwicklung, in Embedded-Systemen oder wenn du einfach mal **grafische Programmierung von Grund auf verstehen willst**.

Wenn du weisst, wie ein Framebuffer funktioniert, **verstehst du auch, wie Bildschirme arbeiten ganz ohne Magie**